

XC-MD: Explainable Cross-Platform Malware Detection Using Stacking Ensemble Learning

Shikha
BTech CSE
(AIML)
School of Computer Science
and Engineering, Dev Bhoomi
Uttarakhand University,
Dehradun, India
shikha.010620@gmail.com

Ananya
BTech CSE
(AIML)
School of Computer Science
and Engineering, Dev Bhoomi
Uttarakhand University,
Dehradun, India
aggarwalananya602@gmail.com

Mukesh Kumar
Assistant Professor
School of Computer Science
and Engineering
Dev Bhoomi Uttarakhand
University, Dehradun, India
mukesh777kumar@gmail.com

Dhajvir Singh Rai
Assistant Professor
School of Computer Science
and Engineering
Dev Bhoomi Uttarakhand
University, Dehradun, India
dhajvirrai123@gmail.com

ABSTRACT— Malware is one of the most damaging and fast-evolving problems we face in cybersecurity today, with attackers regularly attacking Android and Windows systems. Most existing detectors are platform specific and operate as black boxes – this often forces analysts to ignore automated alerts, produce a verdict without any explanation, simply because they cannot verify the system’s reasoning or act on it with confidence – a serious problem in environment, where acting on an unverified verdict carries real consequences.

This paper proposes a unified framework, called XC-MD to handle malware classification for Android APKs and Windows Portable executables under a common shared framework. A stacking ensemble comprising Random Forest, XGBoost, and LightGBM (CNN for Windows) is proposed as base learners. These are then aggregated using an LR meta-learner. SMOTE corrects the class imbalance for training after Mutual Information chooses the top 100 most distinctive features for each platform. To give security teams more transparency to respond to threats with confidence, every detection includes a SHAP global explanation and a LIME per sample explanation.

Evaluated on CICMalDroid 2020 for Android and Ember 2018 for Windows, XC-MD achieved 95.22% accuracy and 99.64% AUC on Android, and 96.74% accuracy with 99.55% AUC on Windows, outperforming all individual baselines. Cross-platform SHAP analysis revealed that Android and Windows malware occupy entirely different feature spaces — confirming that a unified cross-platform framework is not just useful but necessary.

Keywords— *Cross-platform malware, Explainable AI, Stacking ensemble, SHAP, LIME, CICMalDroid, Ember, Cybersecurity.*

I. INTRODUCTION

Malware has shifted from opportunistic single-target attacks into organised campaigns that strike Android and Windows simultaneously, often as part of the same operation [1]. Android holds over 70% of the global mobile market and Windows remains the backbone of enterprise infrastructure — together they represent the most lucrative attack surface available to adversaries [2]. As the boundary between mobile and desktop environments blurs through shared cloud services and interconnected infrastructure, the inadequacy of platform-specific defences has become increasingly difficult to overlook.

For years, signature-based detection was the industry’s answer. Today it struggles — modern malware routinely changes its code structure, hides behind multiple execution layers and delays activation specifically to defeat static inspection [3]. Machine learning stepped in and made genuine progress, with classifiers built on system call sequences, API call patterns and PE file characteristics delivering solid detection rates across both platforms [4]. Yet three fundamental weaknesses persist in nearly every approach proposed so far.

The first is fragmentation — almost no existing system handles both Android and Windows together. Security teams are left running parallel pipelines that share nothing and cannot inform each other [5][6]. The second is class imbalance — malware feature spaces are heavily skewed toward benign samples, and a classifier that simply predicts benign every time can look deceptively accurate on paper [7]. The third and most consequential is opacity — even the strongest models produce a verdict with no explanation attached. Analysts cannot interrogate the reasoning, verify the logic or comfortably stake a response decision on an output they cannot understand [8].

XC-MD addresses all three problems within a single framework. It runs on CICMalDroid 2020 [7] for five-class Android malware classification and Ember 2018 [8] for binary Windows detection, using an identical preprocessing pipeline on both — cleaning, normalization, SMOTE resampling and Mutual Information feature selection down to 100 features per platform.

The core of detection is a stacking ensemble [9] combining Random Forest [10], XGBoost [11] and LightGBM [12] as base learners, with Logistic Regression as the meta-learner that learns how to optimally combine their outputs. Out-of-fold prediction prevents data leakage throughout the stacking process. SHAP provides global feature importance grounded in game theory while LIME explains individual predictions locally [13] — together giving analysts both a population-level view and a per-sample justification for every detection[14][15].

The primary contributions of this work are:

1. We present XC-MD, a unified framework detecting malware on both Android APKs (5-class) and Windows EXEs (binary) using the stacking ensemble architecture — the first such framework to apply systematic dual-layer XAI analysis across both platforms.
2. XC-MD achieves 95.22% accuracy and 99.64% AUC on Android and 96.74% accuracy with 99.55% AUC on Windows, outperforming all individual baseline classifiers on every metric.
3. We integrate SHAP and LIME as a dual-layer XAI mechanism, revealing platform-specific malware behavioral signatures and validating feature importance through cross-method agreement.
4. Cross-platform SHAP analysis demonstrates that Android and Windows malware occupy fundamentally different feature spaces, confirming that architectural unification is achievable even when features are not unified.

The paper is structured as follows. Section 2 describes XC-MD methodology Section 3 presents results and discussion. Section 4 concludes.

II. PROPOSED METHODOLOGY

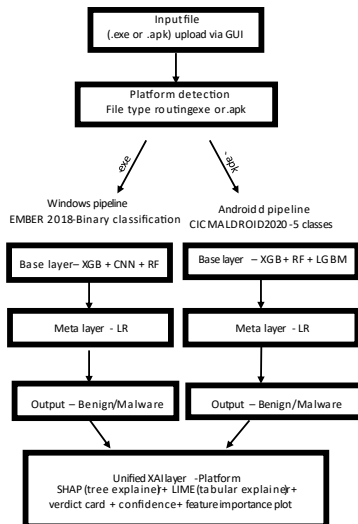


Figure 1: XC-MD cross-platform detection framework architecture

A. Datasets

We evaluate XC-MD on two benchmark datasets. For Android we use CICMalDroid 2020 [16], which contains 11,598 labelled samples across five categories: Adware, Banking Malware, SMS Malware, Riskware and Benign. Features are pre-extracted as 470-dimensional system call frequency vectors. Within the Android dataset, class distribution is uneven — SMS Malware makes up 33.7% of all samples while Adware sits at just 10.8%, the smallest category by a significant margin. We use Ember 2018, a benchmark dataset for Windows that captures PE structural aspects, such as byte frequency distributions, section-level parameters, and import table characteristics, by representing each executable as a 2,381-dimensional vector. The model learns to distinguish between harmful and clean executables in our binary Windows task.

In XC-MD, it is important to understand the meaning of the word 'unified'. Everything above the feature level is shared: Stacking architecture, OOF training strategy, MI feature selection, SMOTE protocol, StandardScaler normalization and the dual-layer SHAP and LIME integration between both layers. The unification is not a representational one; it is an architectural and methodological one; this is the contribution that is being claimed.

B. Preprocessing

To ensure uniformity throughout the framework, both platforms go through the same four-stage pipeline. *Data cleaning*: It involves scanning feature vectors for endless and missing values, replacing them with NaN and removing them row by row.

Label Encoding: Class labels are converted to zero-indexed integers using LabelEncoder. Android's five class labels match integers 0–4, while Windows binary labels are kept as 0 and 1.

Train/Test Split: Stratified sampling is used to divide the data 80/20 while maintaining class proportions in both subsets. To avoid any leakage from preprocessing into the evaluation, the test set is isolated until the final assessment.

Class Imbalance Handling: SMOTE is applied to the Android training set only. It generates synthetic minority samples by interpolating between real instances in feature space, balancing all five classes to 3,123 samples each and growing the training set from 9,278 to 15,615 instances. The test set retains its natural distribution to reflect realistic deployment conditions.

All features are then standardised using StandardScaler fitted on training data only and applied to both sets.

C. Feature Selection

Training on 470 or 2,381 features introduces computational overhead and risks including noisy irrelevant dimensions that reduce generalisation. We apply Mutual Information scoring to rank each feature by its statistical dependency with the class label. Unlike correlation-based methods, MI captures non-linear relationships without distributional assumptions, which suits our heterogeneous feature spaces well.

Scores are computed on the balanced training set and the top 100 features are retained per platform — a 79%

reduction for Android and 96% for Windows. The leading Android features are almost entirely Linux system calls: `getDeviceId`, `mprotect`, `pread64`, `access` and `sendto`. These reflect the behavioural nature of Android malware operating at the kernel level. The top Windows features fall in the PE header and section property index ranges.

D. Stacking Ensemble

The stacking ensemble [9] operates in two layers.

Base Layer: Three classifiers are trained independently on the selected feature set. Random Forest [10] builds an ensemble of decision trees using bootstrap aggregation and random feature subsampling, capturing feature interactions robustly. XGBoost [11] builds trees sequentially where each corrects residuals from the previous, with L1/L2 regularisation preventing overfitting. LightGBM [12] uses leaf-wise tree growth and histogram-based binning for faster training on larger datasets for Android pipeline.

Probability output of each base learner:

Each base learner produces a probability vector over all classes for every input sample:

$$\mathbf{P}^{(k)}(\mathbf{x}_i) = [\mathbf{p}^{(k)}(y=0|\mathbf{x}_i), \mathbf{p}^{(k)}(y=1|\mathbf{x}_i), \dots, \mathbf{p}^{(k)}(y=C-1|\mathbf{x}_i)]$$

These probability vectors are concatenated to form the meta-feature vector:

$$\mathbf{z}(\mathbf{x}_i) = [\mathbf{P}^{(1)}(\mathbf{x}_i) \parallel \mathbf{P}^{(2)}(\mathbf{x}_i) \parallel \mathbf{P}^{(3)}(\mathbf{x}_i)] \quad (1)$$

This gives a 15-dimensional input to the meta-learner for Android (3 models $\times$ 5 classes) and 6-dimensional for Windows (3 models $\times$ 2 classes).

The Logistic Regression meta-learner produces the final prediction:

$$\hat{y} = \mathbf{argmax}_c [\sigma(\mathbf{Wz}(\mathbf{x}_i) + \mathbf{b})] \quad (2)$$

where σ is the softmax function:

$$\sigma(\mathbf{zc}) = \frac{\exp(\mathbf{zc})}{\sum_j \exp(\mathbf{z}_j)} \quad (3)$$

To prevent data leakage, out-of-fold prediction is used. For each fold f , the base learner trains on all other folds and predicts on the held-out fold only:

$$\mathbf{P}^{(k)}(\mathbf{x}_i) = \mathbf{f}_k(\mathbf{x}_i \mid \text{trained on } \mathbf{D} \setminus \mathbf{D}_f), \forall \mathbf{x}_i \in \mathbf{D}_f \quad (4)$$

This ensures the meta-learner trains only on predictions made on unseen data. For Windows, CNN replaces LightGBM as the third base learner, as convolutional filters better capture the byte-level and section-level sequential patterns in Ember's PE structural features, which empirical comparison confirmed outperforms LightGBM on this feature type.

Meta Layer: Logistic Regression serves as the meta-learner. Its simplicity prevents overfitting on the low-dimensional meta-feature space, and it learns optimal combination weights across base learner outputs. It is trained on OOF predictions and evaluated on a separate test meta-feature matrix from base learners trained on the full training set.

Final prediction:

The SHAP value ϕ_i for feature i measures its average marginal contribution across all possible feature subsets:

$$\phi_{i(f,x)} = \sum_{S \subseteq F \setminus \{i\}} \left(\frac{|S|!(|F|-|S|-1)!}{|F|!} \right) [f(S \cup \{i\}) - f(S)] \quad (5)$$

The sum of all SHAP values equals the difference between the model output and the global expected output:

$$\mathbf{f}(\mathbf{x}) = \mathbf{E}[\mathbf{f}(\mathbf{x})] + \sum_i \phi_i \quad (6)$$

Global feature importance is computed as the mean absolute SHAP value across all N test samples:

$$\bar{I}(i) = (1/N) \sum_{j=1}^N |\phi_i(\mathbf{f}, \mathbf{x}_j)| \quad (7)$$

E. Explainability Integration

SHAP — Global Explainability: We use SHAP [17] with TreeExplainer for exact Shapley value computation on our XGBoost base learner. Each feature receives a contribution score per prediction — positive values push toward a class; negative values push away. Averaging absolute SHAP values across test samples produces global feature importance rankings. For Android this produces a (2320, 100, 5) tensor; for Windows a (500, 100) matrix.

LIME — Local Explainability: LIME explains individual predictions by modifying the input sample, observing prediction changes, and fitting a local linear approximation. The linear weights serve as per-prediction feature importance scores. We generate LIME explanations for one correctly classified sample per class — five for Android, two for Windows.

Cross-validating SHAP and LIME results identifies features confirmed by both methods, representing the most reliable discriminative signals in the data independent of either method's assumptions.

F. Evaluation Metrics

We report Accuracy, Weighted Precision, Weighted Recall, Weighted F1 and Macro AUC. Macro AUC computes the arithmetic mean of one-vs-rest AUC scores for each class. It is our primary performance metric.

Macro-AUC averages the individual class AUC scores across all 5 classes, evaluated under a One-vs-Rest scheme:

$$\text{Macro-AUC} = \left(\frac{1}{C} \right) \sum_{c=1}^C \{AUC_c\} \quad (8)$$

where each AUC_c is computed as:

$$AUC_c = \int_0^1 \text{TPR}_{c(FPR_c)} d(FPR_c) \quad (9)$$

Even when class sizes vary significantly, this statistic is still dependable because each class adds the same amount to the average regardless of the number of samples it contains. The remaining metrics are defined as:

$$\text{Accuracy} = \frac{(TP + TN)}{(TP + TN + FP + FN)} \quad (10)$$

$$\text{Precision} = \frac{TP}{(TP + FP)} \quad (11)$$

$$\text{Recall} = \frac{TP}{(TP + FN)} \quad (12)$$

$$\mathbf{F1} = 2 \times \frac{(\text{Precision} \times \text{Recall})}{(\text{Precision} + \text{Recall})} \quad (13)$$

G. Experimental Environment

All tests were carried out using an NVIDIA T4 GPU and 12GB RAM on Google Colab. The software environment comprised Python 3.12, scikit-learn 1.3, XGBoost 2.0, LightGBM 4.0, TensorFlow 2.15, SHAP 0.44 and LIME 0.2.

Table I: Computational performance — Android pipeline (CICMalDroid 2020)

Model	Training Time	Inference (1 sample)	Inference (2,320 samples)
Random Forest	9.14s	43.64ms	65.69ms
XGBoost	12.35s	0.68ms	30.33ms
LightGBM	16.13s	1.21ms	179.60ms
LR Meta-learner	—	—	0.53ms
XC-MD Pipeline	37.62s	—	276.16ms

The complete XC-MD inference pipeline processes 2,320 test samples in 276ms, demonstrating computational feasibility for near-real-time deployment scenarios.

Hyperparameters: RF (n=200, max_features=sqrt), XGBoost (n=200, lr=0.1, depth=6), LightGBM (n=300, lr=0.1, leaves=63), CNN (3×Conv1D, epochs=50, batch=32), LR meta-learner (C=1.0); seed=42 throughout.

III. RESULT AND DISCUSSION

A. Android Results

Table II: Baseline vs XC-MD ensemble performance — Android (CICMalDroid 2020)

Model	Accuracy	F1 (W)	Macro AUC
Random Forest	94.48%	94.50%	99.54%
XGBoost	95.04%	95.05%	99.60%
LightGBM	95.04%	95.04%	99.63%
CNN (parallel)	87.93%	88.07%	97.94%
XC-MD Ensemble	95.22%	95.22%	99.64%

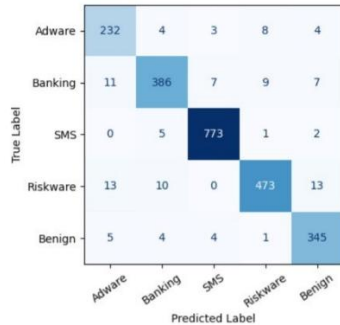


Figure 2: Confusion matrix — XC-MD ensemble on Android test set (CICMalDroid 2020)

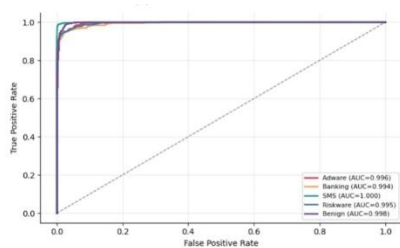


Figure 3: One-vs-Rest ROC curve for Android

Across all five metrics, XC-MD consistently outperformed every individual model. This displays the effectiveness of the proposed stacking technique. The meta-learner reduces errors that no single model could reduce on its own by developing a sense of which base learner to defer based on the input, rather than just averaging outcomes.

The most important outcome in this case is a Macro AUC of 99.64%. It is a reliable figure for actual deployment because it indicates that the ensemble can almost always rank a malicious sample above a benign one regardless of where the classification threshold is set.

Breaking results down by class, SMS Malware came out on top per-class at F1 = 0.986 — not surprisingly, given it had the most balanced training representation after SMOTE and a send-to-heavy system call signature that the models found relatively easy to latch onto. At F1 = 0.906, adware was the opposite. It was the only class in which the ensemble continuously performed poorly, with only 251 test instances and a feature profile that almost resembles Riskware. Less than 4 out of 100 legitimate applications were mistakenly identified, according to the Benign recall of 96.1%. This figure is significant practically because security solutions that generate too many false positives causes the analysts to eventually lose all faith in the system.

To confirm that XC-MD's gains over individual base learners reflect genuine improvement rather than random variation, Wilcoxon signed-rank tests were run across 10-fold cross-validation. XC-MD returned a mean cross-validation accuracy of 95.22% [95% CI: 94.25%–96.19%]. The individual baselines trailed noticeably — RF at 91.77% [90.46%–93.08%], XGBoost at 91.94% [90.54%–93.34%], and LightGBM at 92.33% [90.89%–93.77%]. Every pairwise comparison between XC-MD and a baseline produced $W = 0.0000$ and $p = 0.0020$, placing the results well within the threshold for statistical significance at $\alpha = 0.05$. The ensemble's advantage is not incidental.

B. Windows Results

Table III: Ensemble ablation study — Windows platform (Ember 2018)

Combination	Accuracy
RF + XGBoost + CNN	96.26%
XGBoost + CNN	96.98%
RF + CNN	96.12%
RF + XGBoost	96.29%
XC-MD Final	96.74%

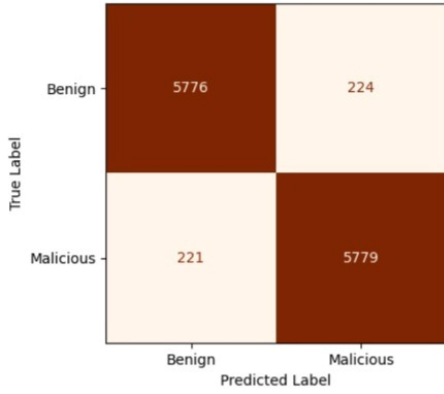


Figure 4: Confusion matrix for Windows

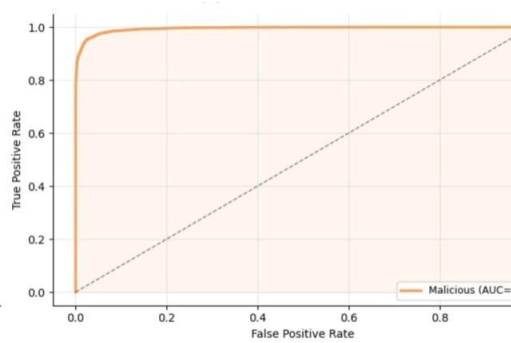


Figure 5: One-vs-Rest ROC curve for Windows

Final metrics: Accuracy 96.74%, Precision 96.53%, Recall 96.97%, F1 96.75%, AUC 99.55%.

At 99.55% AUC, the Windows pipeline demonstrates it can almost perfectly separate clean executables from malicious ones across any decision threshold. Recall of 96.97% carries particular weight in a security context — the cost of missing a real threat far exceeds the cost of a false alarm, making high recall the more operationally critical of the two metrics.

A notable observation from the performance comparison — XGBoost + CNN achieved 96.98% accuracy, marginally above the full ensemble's 96.74%. The full ensemble was selected as the final model based on its superior AUC of 99.55%, which provides threshold-independent discriminability more relevant to real deployment conditions than accuracy alone.

A preliminary robustness check using Gaussian feature perturbation on 50 malicious Windows samples showed XC-MD holding 92% correct classification at $\sigma = 0.5$, a reasonable signal that the model doesn't fall apart under mild noise. That said, Gaussian perturbation is a controlled approximation. Attacks that respect PE structural constraints — section padding, import table inflation are a closer model of how real evasion actually works, and evaluating against those remains on the agenda.

C. Cross-Platform SHAP Analysis

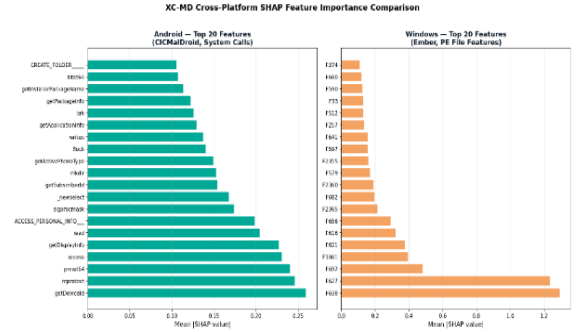


Figure 6: Cross-platform SHAP comparison — Android system calls vs Windows PE indices (zero feature overlap)

Perhaps the most striking outcome of the cross-platform analysis was the total absence of shared features between the two pipelines. On Android, the dominant SHAP contributors were Linux system calls — `getDeviceId` (0.263), `mprotect` (0.249) and `pread64` (0.244). On Windows, two PE structural indices pulled far ahead of everything else — `F638` at 1.292 and `F627` at 1.237. The feature spaces do not overlap at all.

What this tells us is straightforward — malware on these two platforms leaves behind completely different traces at the measurable level, even when the underlying intent is the same. A unified detector built around shared features would simply not work.

What XC-MD shows instead is that unification at the architectural level is both possible and effective. The stacking ensemble, the meta-learner, the OOF training strategy, the SHAP and LIME integration — all of it transfers across platforms without modification, even though the features feeding into it are entirely unrelated. This confirms that architectural unification is achievable independent of feature-level correspondence

D. SHAP and LIME Behavioural Insights

Each Android malware class carries a distinct behavioural fingerprint in the SHAP output. For SMS Malware, the `sendto` system call sits at the top — it is the call responsible for pushing data through network sockets, which is exactly how this class silently dispatches messages and maintains contact with command-and-control infrastructure. Adware uses `getDeviceId` and `getActivePhoneType`, two information-gathering calls that feed the user profile pipeline on which advertising is dependent. The strongest signal for riskware are calls like `getSubscriberId`, which retrieves the device's IMSI number directly for harvesting subscriber identification for exploitation. In order to lay a fraudulent interface over a legitimate banking program at the exact moment, banking malware clusters around `getDisplayInfo`, which monitors screen state.

The LIME results are obtained at the same point. Looking at a single SMS Malware sample, sendto and recvfrom both pushes strongly toward the malware classification while CREATE_FOLDER pushes against it. That last point is telling — SMS Malware has no reason to create files; it only needs to transmit. When two independent explanation methods highlight the same features from completely different directions, the conclusion is hard to argue with: these signals are real, not artefacts of either method's assumptions.

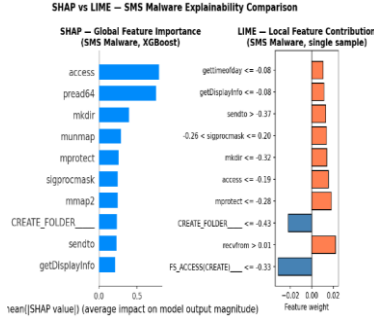


Figure 7: SHAP vs LIME attribution for SMS malware — cross-validated features: access, sendto, sigprocmask

E. Comparison with Prior Work

Table IV sets XC-MD alongside representative prior works, comparing across accuracy, AUC and explainability support.

Table IV: XC-MD vs prior works in malware detection

Work	Platform	Accuracy	AUC	XAI
DREBIN[18]	Android	94.0%	—	—
MalConv [19]	Windows	96.1%	—	—
Warnecke et al. [20]	Android	95.1%	98.2%	SHAP
XC-MD Android	Both	95.22%	99.6%	SHAP + LIME
XC-MD Windows	Both	96.74%	99.55%	SHAP + LIME

XC-MD matches or exceeds prior single-platform works while being the only framework covering both platforms with dual-layer XAI.

There are two recent studies worth comparing XC-MD against. Jeong and Park[21] built a transformer-based NLP model for Android malware detection through Smali code learning — a strong deep learning approach, but single-platform and offering no explainability. Zou et al.[22] took a different route, constructing feature graphs from static PE properties for Windows detection, and while results were competitive, cross-platform capability was

nowhere in the picture. XC-MD does what neither does — detection across both platforms in one unified architecture, with SHAP and LIME explaining every prediction along the way.

IV. CONCLUSION AND FUTURE WORK

This paper presented XC-MD, a unified cross-platform malware detection framework combining stacking ensemble learning with SHAP and LIME explainability to detect and explain malware on both Android and Windows within a single architecture.

On Android, XC-MD achieved 95.22% accuracy and 99.64% Macro AUC across five malware categories using CICMalDroid 2020, outperforming all individual baselines. On Windows, it achieved 96.74% accuracy and 99.55% AUC on the Ember 2018 binary classification task. Both AUC results confirm strong class separability across platforms.

The cross-platform SHAP analysis produced this work's most significant finding — complete feature space separation between platforms. Android malware manifests through Linux system call behavior while Windows malware is captured through PE structural properties. Yet despite operating in entirely separate feature spaces, the same ensemble architecture performed strongly on both. This is the core finding — you do not need shared features to build a shared detector. What you require is a shared design, and XC-MD shows that such a design is both practically and theoretically sound. This distinction is important for anyone developing unified security tools in the future: standardize the architecture, not the features.

The explainability layer added a dimension that pure classification cannot offer. SHAP found class-specific behavioural signatures, such as sendto for SMS transmission, getDeviceId for Adware, and getSubscriberId for Riskware, that directly correspond to known malware behaviours. LIME verified these results at the individual sample level and cross-method agreement proved that they are reliable indicators rather than the product of a single explanation technique.

There are two drawbacks. One is static feature dependency — if malware stays quiet in the sandbox during extraction, it slips through the cracks on both platforms. On Windows, Ember's anonymized feature indices limit how useful SHAP actually is; Android gives you readable system call names, Windows gives you index labels. The future work picks up that explainability gap directly.

Several directions emerge naturally from this work.

1. **Real File Support-** XC-MD currently requires pre-extracted feature vectors as input. Integrating Androguard for APK parsing and LIEF for Windows PE analysis at the front end would allow analysts to submit raw binary files directly, closing the gap between research prototype and deployable security tool without any changes to the existing ensemble architecture.
2. **Dynamic Analysis Integration-** Both datasets rely on static features, which sophisticated malware increasingly evades through runtime obfuscation and delayed

payload activation. Replacing static features with behavioral traces captured inside a sandbox execution environment would make XC-MD significantly more resilient. The stacking architecture is feature-agnostic — it requires no structural changes to consume dynamic features.

3. **Adversarial Robustness-** XC-MD has not been evaluated against an attacker who knows it exists. Windows PE features in particular can be manipulated without breaking executable functionality. Systematic adversarial sample generation followed by adversarial training would quantify and improve robustness under active evasion — a necessary step before any real-world deployment.
4. **Resolving the Windows Explainability Gap-** The interpretability of SHAP results is currently uneven across the two platforms. On Android each feature corresponds to a named Linux system call like read, write so the behavior for prediction is immediately clear. On Windows side however, explanations are harder to read as SHAP output shows names like F1 or F47. Bridging this gap by mapping each index to its actual PE attribute would bring the Windows and Android at same level and transform XC-MD into a truly unified account.
5. **Human-in-the-Loop-Feedback-** Currently, SHAP and LIME provide analysts with explanations of decisions, but they don't get any input in return. By enabling analysts to identify false attributes and implementing those changes back into training, the model's logic would gradually match with expert domain knowledge, increasing XC-MD's credibility over time.

V. REFERENCES

- [1] E. Baghirov, "A comprehensive investigation into robust malware detection with explainable AI," *Cyber Security and Applications*, vol. 3, Dec. 2025, doi: 10.1016/j.csa.2024.100072.
- [2] M. Kumar, Rahmaan, K. A. A. Katiyan, N. M. P. Suwathi, and D. Arivazhagan, "An Enhanced Deep Learning Model for Intrusion Detection System: A Robust Gradient-Based Defence Mechanism," in *2025 2nd International Conference on Circuits, Power and Intelligent Systems (CCPIS)*, IEEE, Sep. 2025, pp. 1–8. doi: 10.1109/CCPIS65231.2025.11234278.
- [3] J. Sahs and L. Khan, "A Machine Learning Approach to Android Malware Detection," in *2012 European Intelligence and Security Informatics Conference*, IEEE, Aug. 2012, pp. 141–147. doi: 10.1109/EISIC.2012.34.
- [4] A. Hussain, M. Asif, M. Bin Ahmad, T. Mahmood, and M. A. Raza, "Malware Detection Using Machine Learning Algorithms for Windows Platform," 2022, pp. 619–632. doi: 10.1007/978-981-16-7618-5_53.
- [5] U. Pradhan, M. K. Navaneeth, and M. N. Aditya, "Malware Analysis Using Hashing and Explainable AI: A Comparative Study of LIME and SHAP Techniques," in *2025 1st International Conference on AIML-Applications for Engineering and Technology, ICAET 2025*, Institute of Electrical and Electronics Engineers Inc., 2025. doi: 10.1109/ICAET63349.2025.10932147.
- [6] S. Singhal, "Intrusion Detection System using Deep Neural Network," in *2025 3rd International Conference on Sustainable Computing and Data Communication Systems (ICSCDS)*, IEEE, Aug. 2025, pp. 297–304. doi: 10.1109/ICSCDS65426.2025.11166868.
- [7] M. M.-H.-Z. Abedin and T. Mehruh, "Comparison of Multiple Classifiers for Android Malware Detection with Emphasis on Feature Insights Using CICMalDroid 2020 Dataset," in *2025 IEEE 7th International Conference on Sustainable Technologies For Industry 5.0 (STI)*, IEEE, Dec. 2025, pp. 1–6. doi: 10.1109/STI69347.2025.11367549.
- [8] M. Şandor, R. M. Portase, and A. Coleşa, "Ember Feature Dataset Analysis For Malware Detection," in *2023 IEEE 19th International Conference on Intelligent Computer Communication and Processing (ICCP)*, IEEE, Oct. 2023, pp. 203–210. doi: 10.1109/ICCP60212.2023.10398693.
- [9] H. Zhu, Y. Li, R. Li, J. Li, Z. You, and H. Song, "SEDMDroid: An Enhanced Stacking Ensemble Framework for Android Malware Detection," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 2, pp. 984–994, Apr. 2021, doi: 10.1109/TNSE.2020.2996379.
- [10] M. S. Alam and S. T. Vuong, "Random Forest Classification for Detecting Android Malware," in *2013 IEEE International Conference on Green Computing and Communications and IEEE Internet of Things and IEEE Cyber, Physical and Social Computing*, IEEE, Aug. 2013, pp. 663–669. doi: 10.1109/GreenCom-iThings-CPSCom.2013.122.
- [11] S. S. Dhaliwal, A.-A. Nahid, and R. Abbas, "Effective Intrusion Detection System Using XGBoost," *Information*, vol. 9, no. 7, p. 149, Jun. 2018, doi: 10.3390/info9070149.
- [12] D. T. Nguyen and S. Lee, "LightGBM-based Ransomware Detection using API Call Sequences," *International Journal of Advanced Computer Science and Applications*, vol. 12, no. 10, 2021, doi: 10.14569/IJACSA.2021.0121016.
- [13] A. M. Salih *et al.*, "A Perspective on Explainable Artificial Intelligence Methods: SHAP and LIME," *Advanced Intelligent Systems*, vol. 7, no. 1, Jan. 2025, doi: 10.1002/aisy.202400304.
- [14] N. Khan, K. Ahmad, A. Al Tamimi, M. M.

- Alani, A. Bermak, and I. Khalil, "Explainable AI-Based Intrusion Detection Systems for Industry 5.0 and Adversarial XAI: A Systematic Review," Dec. 01, 2025, *Multidisciplinary Digital Publishing Institute (MDPI)*. doi: 10.3390/info16121036.
- [15] V. Z. Mohale and I. C. Obagbuwa, "Evaluating machine learning-based intrusion detection systems with explainable AI: enhancing transparency and interpretability," *Front. Comput. Sci.*, vol. 7, 2025, doi: 10.3389/fcomp.2025.1520741.
- [16] A. Galli, V. La Gatta, V. Moscato, M. Postiglione, and G. Sperli, "Explainability in AI-based behavioral malware detection systems," *Comput. Secur.*, vol. 141, Jun. 2024, doi: 10.1016/j.cose.2024.103842.
- [17] R. S K and R. V, "A Survey on Deep Learning-Based Intrusion Detection Systems with SHAP Explainability: Techniques, Challenges, and Future Directions," *International Journal For Multidisciplinary Research*, vol. 7, no. 5, Oct. 2025, doi: 10.36948/ijfmr.2025.v07i05.59221.
- [18] D. Arp, M. Spreitzenbarth, M. Hübner, H. Gascon, and K. Rieck, "Drebin: Effective and Explainable Detection of Android Malware in Your Pocket," in *Proceedings 2014 Network and Distributed System Security Symposium*, Reston, VA: Internet Society, 2014. doi: 10.14722/ndss.2014.23247.
- [19] O. Suci, S. E. Coull, and J. Johns, "Exploring Adversarial Examples in Malware Detection," in *2019 IEEE Security and Privacy Workshops (SPW)*, IEEE, May 2019, pp. 8–14. doi: 10.1109/SPW.2019.00015.
- [20] A. Warnecke, D. Arp, C. Wressnegger, and K. Rieck, "Evaluating Explanation Methods for Deep Learning in Security," in *2020 IEEE European Symposium on Security and Privacy (EuroS&P)*, IEEE, Sep. 2020, pp. 158–174. doi: 10.1109/EuroSP48549.2020.00018.
- [21] W.-S. Jung and J.-P. Park, "Android Malware Detection Method Based on Smali Code Learning Using Natural Language Processing (Transformer) Model," *Journal of the Korea Academia-Industrial cooperation Society*, vol. 25, no. 10, pp. 922–928, Oct. 2024, doi: 10.5762/KAIS.2024.25.10.922.
- [22] B. Zou *et al.*, "Feature Graph Construction With Static Features for Malware Detection," *IET Inf. Secur.*, vol. 2025, no. 1, Jan. 2025, doi: 10.1049/ise2/6687383.